As the number of processors increases, so does the likelihood and performance impact of another problem: that of multiple processors trying to access the same data.

8.2.2. Data contention and cache ping-pong

If two threads are executing concurrently on different processors and they're both *reading* the same data, this usually won't cause a problem; the data will be copied into their respective caches, and both processors can proceed. However, if one of the threads *modifies* the data, this change then has to propagate to the cache on the other core, which takes time. Depending on the nature of the operations on the two threads, and the memory orderings used for the operations, such a modification may cause the second processor to stop in its tracks and wait for the change to propagate through the memory hardware. In terms of CPU instructions, this can be a *phenomenally* slow operation, equivalent to many hundreds of individual instructions, although the exact timing depends primarily on the physical structure of the hardware.

Consider the following simple piece of code:

```
std::atomic<unsigned long> counter(0);
void processing_loop()
{
    while(counter.fetch_add(1,std::memory_order_relaxed)<100000000)
     {
        do_something();
     }
}
```

The `counter` is global, so any threads that call `processing_loop()` are modifying the same variable. Therefore, for each increment the processor must ensure it has an up-to-date copy of `counter` in its cache, modify the value, and publish it to other processors. Even though you're using `std::memory_order_relaxed`, so the compiler doesn't have to synchronize any other data, `fetch_add` is a read-modify-write operation and therefore needs to retrieve the most recent value of the variable. If another thread on another processor is running the same code, the data for `counter` must therefore be passed back and forth between the two